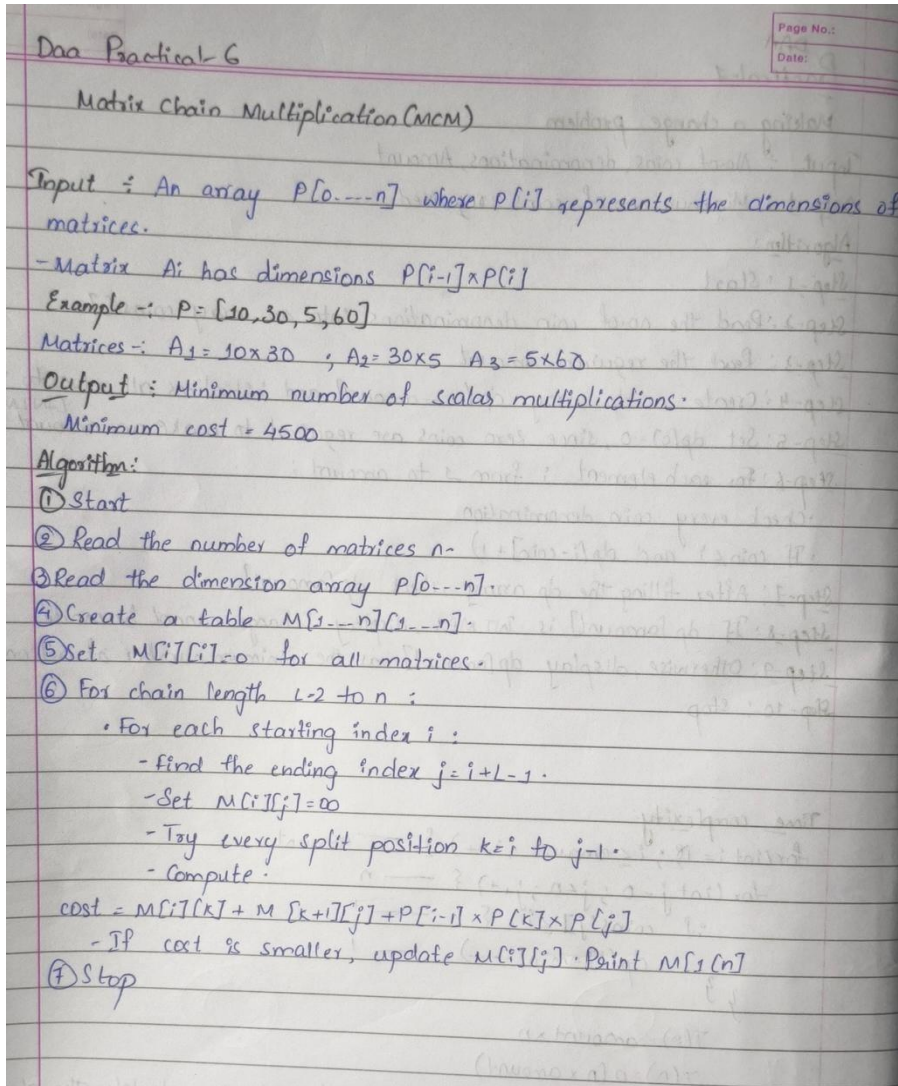


Practical - 6

Aim: Implementation of chain matrix multiplication using dynamic programming.

Algorithm :



Daa Practical-6

Matrix Chain Multiplication (MCM)

Input : An array $P[0 \dots n]$ where $P[i]$ represents the dimensions of matrices.

- Matrix A_i has dimensions $P[i-1] \times P[i]$

Example - : $P = [10, 30, 5, 60]$

Matrices - : $A_1 = 10 \times 30$; $A_2 = 30 \times 5$; $A_3 = 5 \times 60$

Output : Minimum number of scalar multiplications.

Minimum cost = 4500

Algorithm:

- ① Start
- ② Read the number of matrices n .
- ③ Read the dimension array $P[0 \dots n]$.
- ④ Create a table $M[i \dots n][1 \dots n]$.
- ⑤ Set $M[i][i] = 0$ for all matrices.
- ⑥ For chain length $L = 2$ to n :
 - For each starting index i :
 - find the ending index $j = i + L - 1$.
 - Set $M[i][j] = \infty$
 - Try every split position $k = i$ to $j - 1$.
 - Compute.
$$\text{cost} = M[i][k] + M[k+1][j] + P[i] \times P[k+1] \times P[j]$$
 - If cost is smaller, update $M[i][j]$. Print $M[1][n]$
- ⑦ Stop

Program:

```
#include <iostream>
using namespace std;

int main() {
    int n, p[20], c[20][20];
    int i, j, k, d, q, min;

    cout << "Enter number of matrices: ";
    cin >> n;

    cout << "Enter " << n + 1 << " dimensions: ";
    for (i = 0; i <= n; i++) {
        cin >> p[i];
    }

    // Initialize diagonal elements to 0
    for (i = 1; i <= n; i++) {
        c[i][i] = 0;
    }

    // Dynamic Programming
    for (d = 1; d < n; d++) {
        for (i = 1; i <= n - d; i++) {
            j = i + d;
            min = 32767;

            for (k = i; k <= j - 1; k++) {
                q = c[i][k] + c[k + 1][j] + p[i - 1] * p[k] * p[j];

                if (q < min) {
                    min = q;
                }
            }

            c[i][j] = min;
        }
    }

    cout << "Minimum number of scalar multiplications = "
        << c[1][n] << endl;

    return 0;
}
```

Output :

```
Enter number of matrices: 3
Enter 4 dimensions: 4
8
7
10
Minimum number of scalar multiplications = 504
```

Time complexity :

Time Complexity :-

Matrix Chain (P,n)

```

for i=1 to n
    M[i][i] = 0
for L=2 to n
    for i=1 to n-L+1
        j = i+L-1
        M[i][j] = ∞
        for k=i to j-1
            cost = M[i][k] + M[k+1][j] + P[i-1]*P[k]*P[j]
            if cost < M[i][j]
                M[i][j] = cost
return M[1][n]

```

$T(n) = O(n^3)$

Best case :-

Eg :- 3 matrices A=10x20 and B=20x30
Cost = 10x20x30 = 6000

For a general input of n matrices, it always uses three nested loops, so the complexity is written as $O(n^3)$.

$\therefore$ Best case = $O(n^3)$

Average Case :- A = 4x6 B = 6x7 C = 7x3
(A B) C = 252 A (B C) = 198

The algorithm calculates both possibilities before choosing the minimum. Therefore, the loop still executes completely given $O(n^3)$.

$\therefore$ Average Case = $O(n^3)$

Worst case :- A = 10x20 B = 20x30 C = 30x40 D = 40x50

* Even though there are more parenthesizations, the DP algorithm systematically evaluates all required subproblems.

$\therefore$ Worst case = $O(n^3)$

Conclusion :

Matrix Chain Multiplication demonstrates the power of Dynamic Programming in optimizing complex computational problems. By systematically solving subproblems and storing their results, the algorithm successfully reduces the time complexity from an exponential brute-force approach to an efficient $O(n^3)$ polynomial time. This optimal parenthesization strategy minimizes the total number of scalar multiplications required, making it an essential technique for resource-constrained matrix operations.

